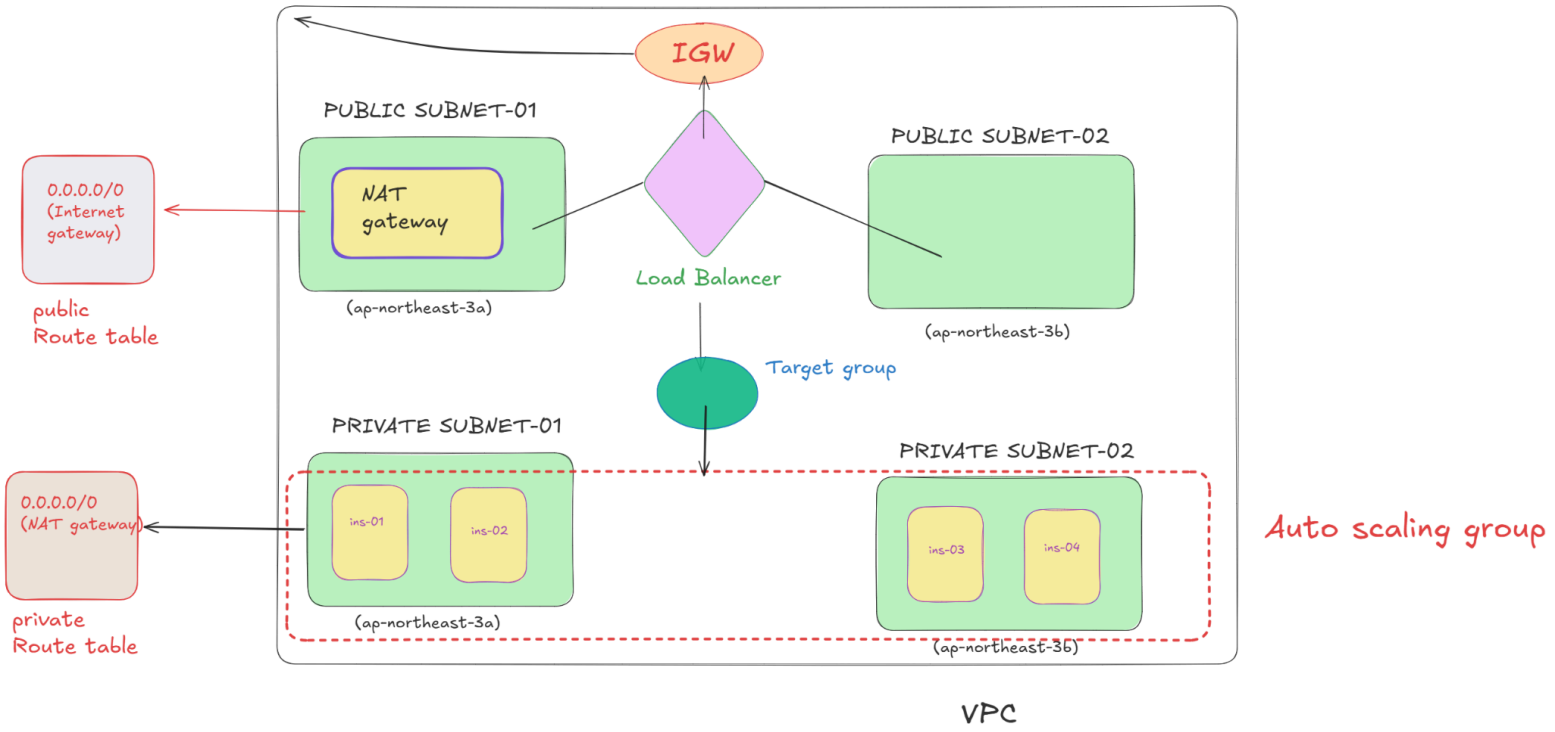
VPC Project

Implementation of Multi-AZ Scalable Infrastructure using AWS VPC, ALB, and NAT Gateway

VPC Architecture Diagram



Components Used in Project

1. Private and public subnets
2. Route table
3. Internet Gateway
4. NAT gateway
5. EC2 instances

6. Auto Scaling Group
7. Elastic Load Balancer

Steps to implement ⬇️

1. Create a VPC network and setup subnets and attach route tables for public and private subnets

- ❖ Go to VPC service in AWS and click on create VPC
- ❖ Provide the name for VPC and set CIDR block (ex: 10.0.0.0/16)
- ❖ Click on next

To create subnets

- Go to subnets in VPC dashboard and click on create subnets
- Provide the subnet name and select one availability zone for each subnet
- Set the subnet CIDR block (ex: 10.0.0.0/22)
- Click on add subnet for adding more subnet and once adding click on create subnets

Create 4 subnets for /22 CIDR block

Subnet name	Subnet CIDR	Availability zone
public-subnet-01	10.0.0.0/22	ap-northeast-3a
public-subnet-02	10.0.4.0/22	ap-northeast-3b
private-subnet-01	10.0.8.0/22	ap-northeast-3a
private-subnet-02	10.0.12.0/22	ap-northeast-3b

To create route tables

- ❖ Once after creating subnets go to route tables and click on create route table
- ❖ Provide route table name and click on create route table

(Create 2 route tables one as public RT and one as private RT)

Attach subnets for created route tables

- Once after route table is created , select the public RT and click on actions and click on edit subnet associations and select all public subnets
- In route table again click on private route table and click on actions and click on edit subnet associations and select all private subnets

Create and attach Internet Gateway to VPC

- In VPC dashboard, click on Internet gateway and click on create Internet gateway
- Select the VPC in which we want to create Internet gateway
- Click on create internet gateway

To attach Internet Gateway to VPC

- Once after internet gateway is created click on actions and click on attach to VPC

Attach public route table to Internet Gateway

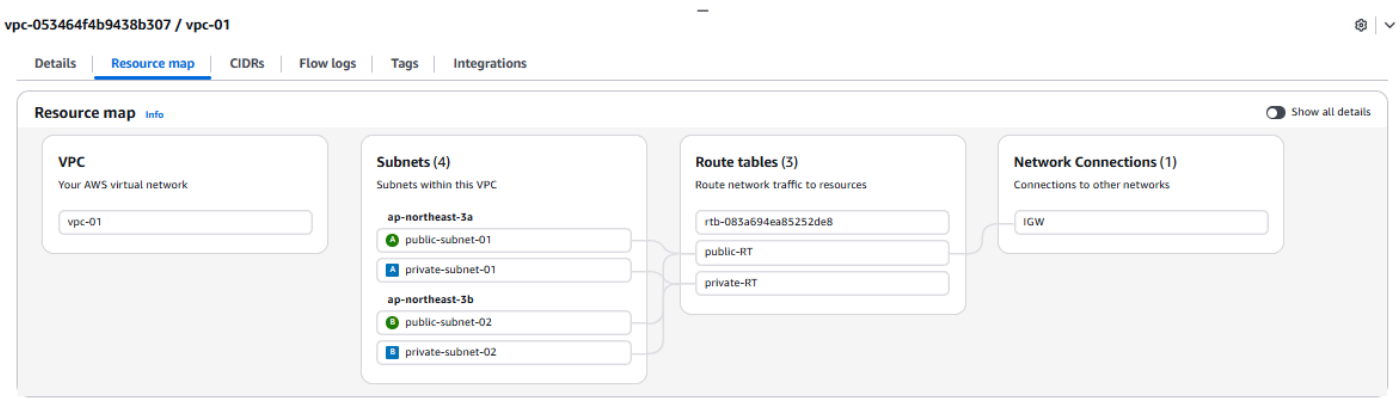
- Once after the Internet gateway is created go to route tables and click on public route tables.
- Click on actions and click on edit routes
- In source add “0.0.0.0/0” and in destination add “Internet gateway” and select the created internet gateway and save changes

Create NAT gateway inside public subnet and attach private route table to Internet gateway

- In VPC dashboard , click on NAT gateway and click on create NAT gateway
- Give the name for NAT gateway and select zonal and select public subnet
- Allocate one elastic IP and click on create NAT gateway

- ❖ Once after NAT gateway is created , go to private Route table and select the private route table and click on actions and click on edit routes
- ❖ Add source as “0.0.0.0/0” and destination as “NAT gateway” and select the created NAT gateway

Resource map of entire VPC architecture once after all components created and attached



Creating Auto Scaling Group and Load Balancers inside created VPC for private subnets

Launch one EC2 instance and host an application in default VPC

- Create one EC2 instance by selecting ubuntu as an OS
- Select VPC as default VPC and allow HTTP (80) and SSH (22) port in security group
- Click on launch instance
- Once after launching the instance connect it and host one application

To host an application inside EC2

- Install unzip and install nginx and download a files from **“tooplate template”** website

Script:

sudo apt update

sudo apt install unzip

sudo apt install nginx

wget -O app.zip <download-link-of-template>

unzip app.zip

- Run the script - **bash script_name.sh**

- Copy the extracted folder to nginx path (**/var/www/html**)

Command: sudo cp -r <extracted-folder-name>/* /var/www/html

- Expose the application by copying the public IP of instance and paste it in browser with 80 port number

Sample URL: http://<public-IP-instance>:80

❖ **Create an image via AMI for the launched instance which is hosting application**

- Once after instance is launched successfully, go to instance dashboard and click on the instance and click on action
- Click on images and templates and click on create image and provide the image name and click on create image
- Go to **AMI** and verify the image is in available state and successfully created

❖ **Create a Launch Template once after image is created**

- Once after image is created go to **Launch Template** in instance dashboard and click create Launch template

- Give the template name and tick the option which provides auto scaling guidance

- Select the image which has been created under My AMI

- Choose the instance type as “t2.micro”

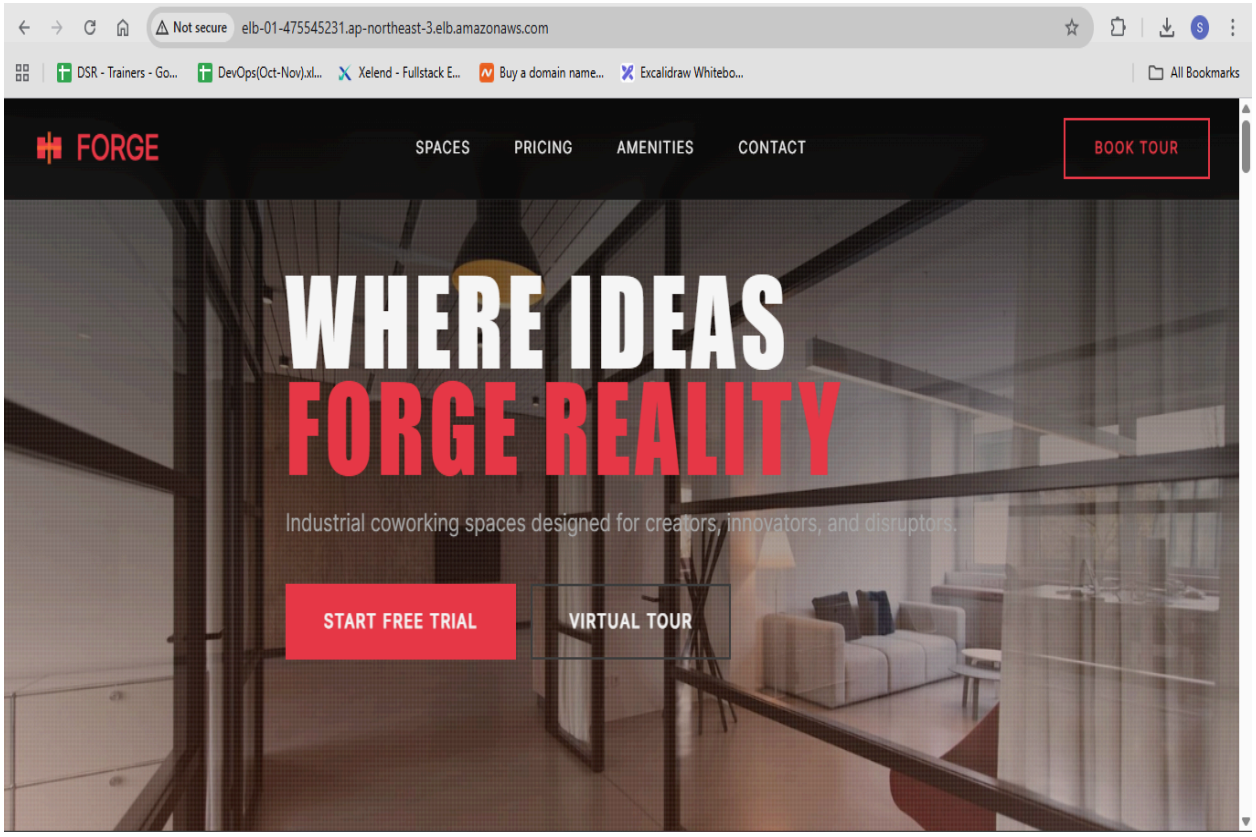
- Create one key pair

- In network settings under **security group** select **default** and select security group for create VPC by referring the VPC ID of created VPC
- Click on launch template

❖ **Create one Auto Scaling Group and attach Load Balancer**

- In instance dashboard, click on Auto scaling group
- Click on create auto scaling group and provide the name for auto scaling group
- Choose the created template and click on next
- In network and availability zones select **"2 private subnets"** which are created in different availability zones and select balanced best effort
- Click on next
- In Load Balancer option click on attach to a new Load Balancer
- Give the load balancer name and select load balancer type as application load balancer
- Under load balancer scheme select as **Internet facing**
- Select subnets as public in availability zones
- Under target and listener click on create a new target group and give target group name
- Click on next
- **Select desired instance as 1 and minimum instance as 1 and maximum instance as 3**
- Under scaling policy select target tracking scaling policy
- Select metric as **CPU utilization** and select **target value as 5**
- Click on next skip the notification and tags part and click on next
- Make sure load balancer is in active state and created and auto scaling is also created and desired instance is present in instance dashboard
- Make sure from load balancer DNS endpoint the application is accessible

- If its not accessible go to security group in load balancer and add HTTP(80) and SSH (22) port and then try to access
-



Application accessed via Load Balancer

- **Launch one EC2 instance in public subnet of created VPC and ssh for private instance for scaling the EC2 instance in private subnet**
  - Go to EC2 dashboard , click on launch instance
  - Select OS as ubuntu and instance type as t2.micro
  - Attach the created key pair
  - In network settings click on edit and select the created VPC
  - Choose anyone public subnet and enable public IP
  - Choose security group as default which is present in created VPC
  - Click on launch instance

-

❖ **Connect to the instance which is created and do SSH into private instance which is present in created VPC**

- Connect to the created EC2 instance which has public IP address
- Create one file as “key.pem” and add private keypair content which is attached for the instance
- Give read permission for the “key.pem” file - **chmod 400 key.pem**
- SSH into another instance
- **Command: ssh-keygen -i “key.pem” ubuntu@<private-IP-desired-instance>**
- Make sure you have logged into private instance (desired instance)

❖ **Increase the load on desired instance to scale up the instance using stress**

- Once after the desired instance is connected via SSH , install stress command
- **Command: sudo apt update && sudo apt install stress -y**
- Provide stress command to increase the load
- **Command: stress –cpu 2 –timeout 300**

Verify for few minutes and observe whether instances are scaled up in the EC2 dashboard

Snapshot after scaling the instance via stress command:

